

Memoria final detallada del proyecto

TaskMaster – David Felipe Rivera Rincón

1. Introducción

TaskMaster es un proyecto desarrollado en Java orientado a la gestión de tareas, usuarios, categorías y estados. El objetivo principal ha sido construir una aplicación de escritorio basada en consola que permita organizar información de manera estructurada, validada y mantenible. A lo largo del proyecto se han aplicado contenidos de varios módulos del ciclo formativo, integrando programación, documentación, análisis de riesgos, pruebas, diseño de interfaz y control de versiones.

Este proyecto no solo se centra en el funcionamiento básico de una aplicación de gestión, sino también en la calidad del software: estructura del código, validación de datos, pruebas unitarias, documentación técnica, refactorización y propuesta de una interfaz visual.

2. Objetivos del proyecto

Los objetivos principales del proyecto TaskMaster han sido los siguientes:

- Desarrollar una aplicación funcional para la gestión de tareas.
 - Organizar la información mediante entidades relacionadas entre sí.
 - Aplicar validaciones para evitar errores de entrada de datos.
 - Separar responsabilidades para mejorar la mantenibilidad del código.
 - Diseñar una interfaz web de presentación clara y responsive.
 - Implementar pruebas unitarias con JUnit para validar la lógica principal.
 - Documentar el código con Javadoc.
 - Detectar errores, corregirlos y registrar las mejoras realizadas.
 - Analizar los riesgos del desarrollo y proponer medidas preventivas.
 - Entregar el proyecto de forma ordenada en GitHub.
-

3. Descripción general de la aplicación

TaskMaster permite gestionar distintos elementos del sistema:

- **Usuarios:** personas que pueden estar asociadas a tareas.
- **Categorías:** agrupaciones temáticas para clasificar tareas.
- **Estados:** situación en la que se encuentra una tarea.
- **Tareas:** elemento principal del sistema, asociado a usuario, categoría y estado.

La aplicación se maneja desde un menú de consola que permite realizar operaciones como:

- crear registros,
- mostrar listados,
- buscar elementos por ID,
- editar tareas,
- eliminar tareas,
- marcar tareas como completadas,
- filtrar tareas por categoría,
- mostrar tareas de un usuario.

Además, como parte del reto de interfaz, se ha diseñado una propuesta web que representa de forma visual el proyecto y la identidad del equipo.

4. Estructura general del proyecto

La estructura del proyecto se ha organizado en varios paquetes y archivos para facilitar su comprensión y mantenimiento.

4.1. Paquete modelo

Contiene las clases principales del dominio del sistema:

- Usuario
- Categoría
- Estado
- Tarea

Estas clases representan los datos básicos que maneja la aplicación.

4.2. Paquete servicio

Incluye la lógica de negocio y la gestión de las entidades:

- GestorUsuarios
- GestorCategorias
- GestorEstados
- GestorTareas

Los gestores se encargan de almacenar, buscar, crear, editar y eliminar elementos.

4.3. Paquete util

Contiene clases auxiliares que ayudan en tareas comunes:

- EntradaConsola
- Validador
- DatosIniciales

Estas clases mejoran la organización del código y evitan repeticiones.

4.4. Paquete vista

Incluye la interfaz de consola:

- MenuConsola

Es la clase encargada de mostrar los menús y conectar al usuario con la lógica del sistema.

4.5. Carpeta test

Contiene las pruebas unitarias desarrolladas con JUnit 5:

- pruebas para Validador
- pruebas para GestorUsuarios
- pruebas para GestorCategorias
- pruebas para GestorEstados
- pruebas para GestorTareas

4.6. Carpeta docs

Incluye la documentación del proyecto en formato Markdown:

- memoria del reto 5,
- memoria del reto 6,
- informe de errores y mejoras,

- diagnóstico de riesgos,
 - memoria final del proyecto.
-

5. Desarrollo por fases del proyecto

5.1. Fase inicial: planificación y análisis

En la primera fase se definieron las funcionalidades necesarias del sistema. Se decidió que la aplicación debía ser capaz de gestionar tareas y entidades asociadas de forma sencilla. Para ello, se identificaron las clases principales y las relaciones entre ellas.

Se establecieron las bases del proyecto teniendo en cuenta:

- la necesidad de una estructura clara,
- la separación entre lógica de negocio y presentación,
- la posibilidad de ampliar el sistema más adelante,
- la futura implementación de pruebas y documentación.

Esta fase fue importante porque permitió organizar el trabajo antes de comenzar la implementación.

5.2. Fase de diseño del modelo de datos

En esta fase se diseñaron las clases del dominio y sus relaciones. Se tomó como base que una tarea debía estar vinculada a:

- un usuario,
- una categoría,
- un estado.

De esta forma se consiguió un modelo más realista y útil para la gestión de información. Cada clase del modelo incorpora atributos, constructores, getters, setters y métodos de representación textual.

Relación entre entidades

- Un usuario puede tener varias tareas.
- Una categoría puede agrupar varias tareas.
- Un estado puede aplicarse a múltiples tareas.
- Cada tarea está asociada a un único usuario, categoría y estado.

5.3. Fase de implementación inicial

En esta etapa se desarrolló la lógica básica del sistema. Se implementaron las clases modelo y los primeros gestores para almacenar los datos en memoria. La aplicación comenzó a funcionar mediante un menú de consola desde el cual se podían realizar operaciones principales.

Se priorizó la funcionalidad básica:

- creación de elementos,
- listado de elementos,
- búsqueda por identificador.

Posteriormente se fueron añadiendo funcionalidades más avanzadas como la edición, la eliminación y el filtrado.

5.4. Fase de ampliación y mejora del sistema

A medida que el proyecto avanzó se detectó la necesidad de mejorar la organización interna del código. Para ello se introdujeron nuevas clases auxiliares y se refactorizó parte del menú.

Mejoras aplicadas

- creación de `EntradaConsola` para centralizar la lectura de datos,
- creación de `Validador` para agrupar comprobaciones,
- creación de `DatosIniciales` para cargar datos de ejemplo,
- simplificación de métodos largos,
- separación más clara de responsabilidades.

Estas decisiones permitieron que el código fuera más limpio, más reutilizable y más fácil de mantener.

6. Interfaz de consola

La interfaz principal del sistema es un menú de consola. Desde él se puede navegar entre distintas opciones de gestión. Se diseñó con una estructura jerárquica clara:

- menú principal,
- submenú de usuarios,

- submenú de tareas,
- submenú de categorías,
- submenú de estados,
- submenú de consultas.

Ventajas de esta interfaz

- es sencilla de usar,
- no requiere conocimientos técnicos avanzados,
- permite organizar las funcionalidades por bloques,
- favorece la interacción directa con el usuario.

Funcionalidades del menú

- crear usuario,
- mostrar usuarios,
- buscar usuario por ID,
- crear tarea,
- editar tarea,
- eliminar tarea,
- mostrar tareas,
- marcar tarea como completada,
- filtrar tareas por categoría,
- mostrar tareas de un usuario,
- crear y consultar categorías,
- consultar estados.

7. Propuesta de interfaz web

Además de la aplicación de consola, se diseñó una propuesta de interfaz web para la presentación del equipo y del proyecto. Esta propuesta cumple los requisitos del reto 5 y aporta una visión más visual y moderna de TaskMaster.

Elementos incluidos

- al menos dos vistas,

- cabecera,
- footer,
- formulario de contacto,
- uso de Flexbox,
- propiedad de posicionamiento (fixed, sticky o relative),
- animación,
- diseño responsive.

Objetivo de la web

La web no sustituye a la aplicación de consola, sino que la complementa como propuesta visual. Sirve como presentación del equipo y como demostración de cómo podría evolucionar TaskMaster a una interfaz gráfica completa.

8. Diseño de pruebas

Uno de los aspectos más importantes del proyecto ha sido la validación del sistema mediante pruebas. Para ello se definieron casos de prueba sobre los flujos críticos de la aplicación.

Casos de prueba diseñados

- crear una tarea con fecha límite anterior a la fecha de creación,
- crear usuarios con email no válido,
- introducir textos vacíos en campos obligatorios,
- buscar tareas o entidades inexistentes,
- eliminar elementos que no existen,
- crear registros duplicados,
- comprobar el funcionamiento de los listados.

Objetivos de las pruebas

- verificar que las validaciones funcionan correctamente,
- evitar errores de lógica,
- comprobar el comportamiento de los gestores,
- garantizar que la aplicación responde de forma coherente ante entradas válidas e inválidas.

9. Implementación de pruebas unitarias con JUnit

Se desarrollaron pruebas unitarias con JUnit 5 para las clases más importantes del sistema. Estas pruebas permiten comprobar automáticamente que la lógica del programa se comporta como se espera.

Clases probadas

- Validador
- GestorUsuarios
- GestorCategorias
- GestorEstados
- GestorTareas

Aspectos comprobados

- validación de texto no vacío,
- validación de correo electrónico,
- validación de fechas,
- creación de usuarios,
- control de duplicados,
- búsqueda por ID,
- eliminación de tareas,
- existencia de estados iniciales,
- creación de categorías.

Ventajas de las pruebas

- detectan fallos antes de la entrega,
- facilitan el mantenimiento,
- sirven como documentación ejecutable,
- reducen el riesgo de introducir errores nuevos al modificar el código.

10. Errores detectados y corregidos

Durante el desarrollo se detectaron varios errores o puntos débiles. Estos problemas fueron corregidos para mejorar la estabilidad del sistema.

10.1. Validación insuficiente de fechas

Se permitió inicialmente crear tareas con fecha límite anterior a la de creación.

Corrección: se añadió una validación específica en la clase Validador.

10.2. Campos vacíos sin control

Algunas entradas de texto podían quedar vacías.

Corrección: se creó un método para comprobar que el texto no esté vacío.

10.3. Duplicados en entidades

Era posible insertar elementos con identificadores repetidos.

Corrección: se revisó la lógica de los gestores para evitar duplicados.

10.4. Menú demasiado grande

La clase MenuConsola concentraba demasiada lógica.

Corrección: se separó la lectura de datos en EntradaConsola y se reorganizaron los métodos.

10.5. Falta de mensajes claros

Algunos mensajes al usuario no eran suficientemente descriptivos.

Corrección: se revisaron los textos mostrados por consola para hacerlos más comprensibles.

11. Refactorización y mejora del código

La refactorización ha sido una parte importante del proyecto. Se han aplicado cambios para reducir la complejidad y mejorar la legibilidad.

Mejoras realizadas

- separación de responsabilidades,
- reducción de código repetido,
- centralización de validaciones,
- centralización de la entrada por consola,
- mejora del orden de las clases,

- simplificación de operaciones repetitivas.

Resultado

El código final es más fácil de leer, mantener y ampliar.

12. Documentación técnica con Javadoc

Se añadió documentación técnica mediante Javadoc en las clases y métodos principales del proyecto. Esta documentación facilita la comprensión del sistema por parte de otros desarrolladores o del propio autor en el futuro.

Etiquetas utilizadas

- @param
- @return
- @throws
- @author
- @version

Finalidad

- documentar el propósito de cada clase,
 - explicar el funcionamiento de los métodos,
 - facilitar la reutilización del código,
 - mejorar el mantenimiento del proyecto.
-

13. Diagnóstico de riesgos

También se elaboró un análisis de riesgos para identificar posibles problemas durante el desarrollo del proyecto.

Riesgos identificados

- no cumplir los plazos de entrega,
- pérdida de información,
- uso inadecuado de equipos informáticos,
- errores en el código no detectados a tiempo,
- fallos en la gestión de información o consultas.

Medidas preventivas

- planificación del trabajo,
- copias de seguridad frecuentes,
- uso responsable del equipo,
- pruebas unitarias,
- revisión periódica del código,
- uso de GitHub como control de versiones.

Este análisis ayuda a entender que el desarrollo de software no solo consiste en programar, sino también en prever problemas y gestionarlos adecuadamente.

14. Relación con los módulos del ciclo

El proyecto TaskMaster ha integrado contenidos de varios módulos:

Programación

- desarrollo en Java,
- clases, objetos, métodos, estructuras de control,
- gestión de listas y lógica de negocio.

Entornos de Desarrollo

- uso de JUnit,
- documentación con Javadoc,
- refactorización,
- pruebas y mejora del código.

Bases de datos

- modelado conceptual de la información,
- relación entre entidades,
- idea de persistencia y optimización de consultas.

Lenguajes de Marcas

- diseño de la interfaz web,
- uso de HTML y CSS,

- estructura de páginas,
- responsive design,
- Flexbox y animaciones.

IPE I

- análisis de riesgos,
- organización del trabajo,
- prevención de errores,
- responsabilidad en el uso de herramientas y recursos.

15. Gestión del proyecto y uso de GitHub

Todo el trabajo se ha organizado y entregado mediante GitHub. Esto ha permitido:

- mantener un historial de cambios,
- guardar copias seguras del proyecto,
- subir documentación y código de forma ordenada,
- facilitar la entrega final al profesorado.

El uso de GitHub también ha servido como práctica real de trabajo colaborativo y control de versiones, algo fundamental en proyectos de desarrollo profesional.

16. Resultados obtenidos

Al finalizar el proyecto se ha conseguido:

- una aplicación funcional de gestión de tareas,
- una estructura de código más limpia,
- validaciones mejoradas,
- pruebas unitarias implementadas,
- documentación técnica con Javadoc,
- una interfaz web de presentación,
- una memoria técnica completa,
- un informe de errores y mejoras,

- un diagnóstico de riesgos.
-

17. Conclusión final

TaskMaster ha sido un proyecto muy completo que ha permitido aplicar conocimientos de varios módulos de forma integrada. A lo largo del desarrollo se ha trabajado no solo en la funcionalidad del software, sino también en su calidad, documentación, validación, presentación visual y análisis de riesgos.

El resultado final es una base sólida, organizada y profesional que demuestra el proceso completo de desarrollo de una aplicación desde la planificación hasta la entrega final.

18. Enlace al repositorio

URL pública del repositorio GitHub:

<https://github.com/felipedaw1/taskmaster>